Manual del programador v1.0.0 — Juego de Naves en Consola

Alumno: Federico Mattia — LPR 5° 3° A-B — E.E.S.T. N° 1 Eduardo Ader.

Documento técnico del archivo `src/main.cpp`.

1. Descripción general

Simulador de combate espacial en la consola de Windows, escrito en C++ con Programación Orientada a Objetos. El jugador controla una nave, esquiva asteroides y los destruye disparando. El programa se compone de tres clases (Nave, Asteroide y Bala) y un bucle principal.

2. Dependencias y entorno

Elemento	Detalle
Sistema operativo	Windows 10/11 (usa la API de consola de Windows)
Compilador	g++ (MinGW / MSYS2)
`windows.h`	gotoxy (posicionar el cursor), ocultar el cursor y Sleep
`conio.h`	`_kbhit()` y `_getch()` para leer teclas sin Enter
Otras	`iostream, vector, ctime, cctype`

3. Clases

Nave (jugador). Sección `private:` `x, y` (posición), `vidas` y `puntaje`. Sección `public:` constructor `Nave(int _x, int _y, int _vidas)`, getters (`getX, getY, getVidas, getPuntaje`), `reducirVida()` (nunca baja de 0), `sumarPuntos(int)` (solo acepta valores positivos), `pintar(), borrar()` y `mover(char)` (limita la nave a la pantalla).

Asteroide. Atributos privados `x, y`. `mover()` lo baja una fila por ciclo y, al salir por abajo (y > 23), lo reubica arriba en una columna aleatoria.

Bala. Atributos privados `x, y`. `mover()` la sube una fila; `fueraDeRango()` indica cuándo debe eliminarse (y < 4).

4. Flujo del programa

(1) Se pide el nombre del piloto y se limpia la pantalla. (2) Se crean la nave (35, 20, 3 vidas), 5 asteroides y un vector de balas, ambos con memoria dinámica (`new`). (3) Bucle principal mientras no haya `gameOver`: dibuja el HUD, lee teclado, mueve las balas, mueve los asteroides y evalúa las colisiones, y espera 30 ms con `Sleep(30)`. (4) Al terminar se libera la memoria y se muestra la pantalla de fin con el puntaje final.

5. Colisiones

Asteroide-Nave: el asteroide cae dentro del rectángulo de 5x3 de la nave; se resta una vida y el asteroide se reubica. Bala-Asteroide: coinciden la columna y la fila (o la fila siguiente); se elimina la bala, se suman 10 puntos y el asteroide se reubica. Cada asteroide reemplazado se libera con `delete` antes de crear el nuevo.

6. Funcionalidad extra: sistema de puntaje

Se agregó el atributo privado `puntaje`, inicializado en 0 en el constructor; los métodos públicos `sumarPuntos(int)` y `getPuntaje()`; la línea del HUD `PUNTAJE`; la llamada `jugador.sumarPuntos(10)` en la colisión bala-asteroide; y el puntaje final en la pantalla de fin.

7. Compilación

```
g++ src/main.cpp -o src/juegodenaves.exe
.\src\juegodenaves.exe
```

8. Cómo extender el juego

Para nuevas funciones: agregar el atributo en la sección `private:` de la clase correspondiente, inicializarlo en el constructor y exponerlo solo mediante métodos `public:` que validen los valores. Evitar acceder a los atributos desde `main()` de forma directa.